

SELECTIVE TICKET ROUTING

You choose the precision. The data returns the coverage.

A response to the Astana Hub technology task for TechDrive LLC, Almaty, and the working code behind it.

Thomas Hotton · August 2026

What TechDrive asked for

1

Automatic classification

NLP microservice classifies and routes every incoming ticket in seconds

2

A confidence gate at 85%

Below it, the ticket goes to manual moderation

3

Routing accuracy above 95%

Stated as the headline target

4

Analytics that are 100% reliable

So management can find real bottlenecks

**The last two cannot
both be true.**

Not because the target is ambitious.
Because of what the gate does to the
tickets it holds back.

The manual path is the dirty one

A confidence gate is sold as the safeguard on data quality: uncertain tickets go to a human, so the tags stay clean. But the human path is the least accurate path in the system, and it is the one the brief itself blames for bad tags.

All manual	75.0%
Full automation, no gate	92.2%
Best achievable, gate at optimum	95.2%
What the brief asked for	unreachable 100%

Past the optimum, raising precision makes the analytics worse.

Every ticket withheld is handed to a process less accurate than the model was on it.

So I built the mechanism and measured it

Rather than propose a method on the strength of its literature, the threshold-setting mechanism was implemented end to end on a public corpus with the same long-tailed category shape as a support taxonomy.

1

Rules

Deterministic pre-filter on the obvious, high-volume patterns.
Zero model cost, fully auditable.

2

Retrieval

Ticket embedded, compared to the labelled archive, category by distance-weighted vote. Every decision names the archive documents behind it.

3

Adjudication

A model call only when the neighbour vote is too close to call. The expensive path, on a minority by design.

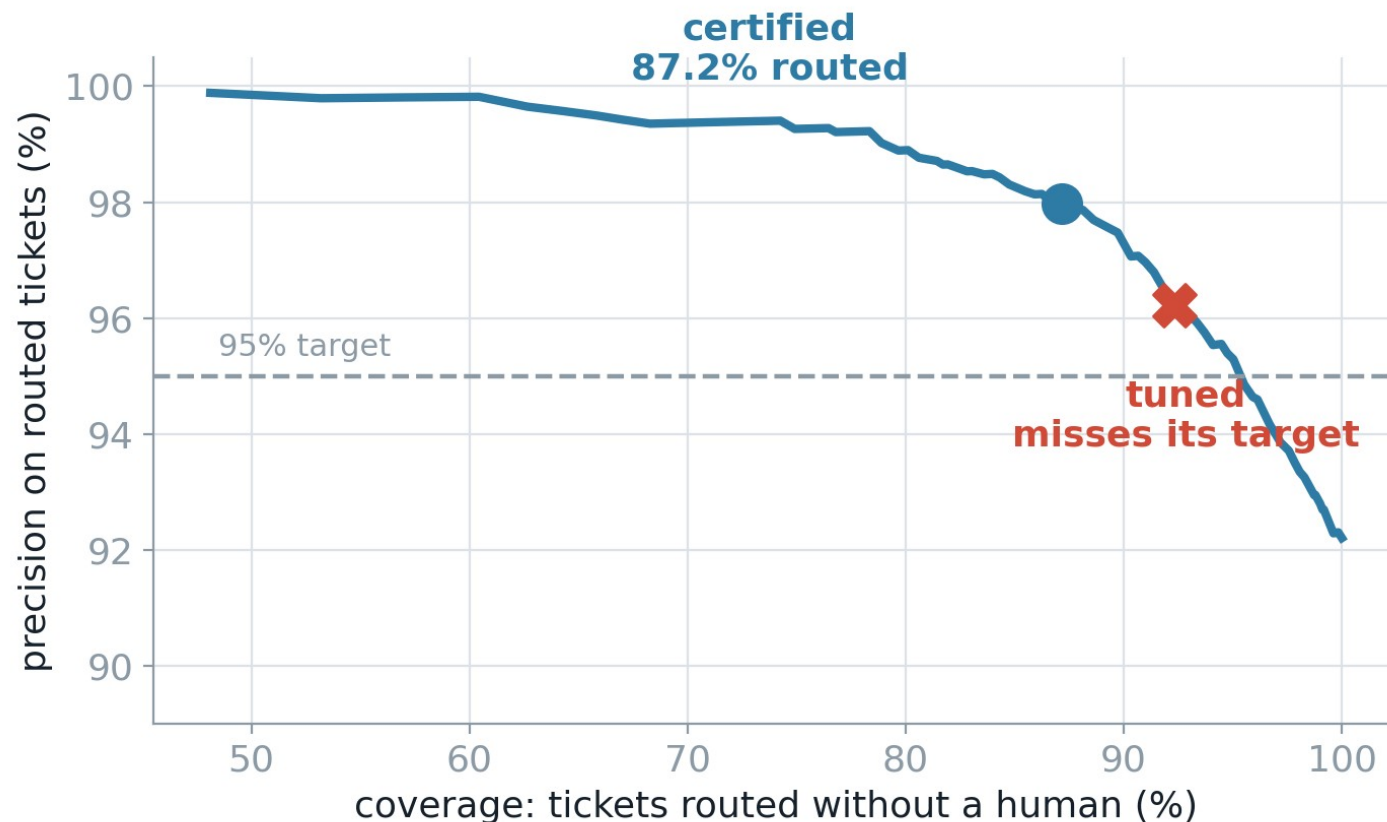
4

Certification

The confidence threshold is not tuned. It is certified by risk control, and refused when the data cannot support it.

Reuters-21578, 53 categories. Top 5 carry 79% of the volume; the median category holds 14 documents. That shape is the point.

Precision is a setting. Coverage is the bill.



CERTIFIED

$\theta = 0.595$ · error $\leq 5\%$

Ask for 95% precision and 87.2% of tickets can be routed automatically.

Ask for 99% and the instrument refuses: no threshold on the grid can be certified from the calibration data.

The refusal is the feature.

The obvious method misses its own target

Sweep the cutoff on held-out data, keep the one whose error rate clears the target. It is what most people would do. Over 15 independent splits:

RISK CONTROL

0%

of trials breached the target
mean coverage 90.9%

THRESHOLD TUNING

67%

of trials breached the target
mean coverage 95.0%

Tuning shows the better coverage. It buys it by not paying for a guarantee, and the failure is invisible on any single run.

One number over 43 categories hides the damage

Every figure so far is an aggregate. A support desk does not experience an aggregate: it experiences one queue per team.

1/43

categories can carry their own certified guarantee at the 95% target

805

calibration items held by the one that qualifies. The median of the rest holds 7.

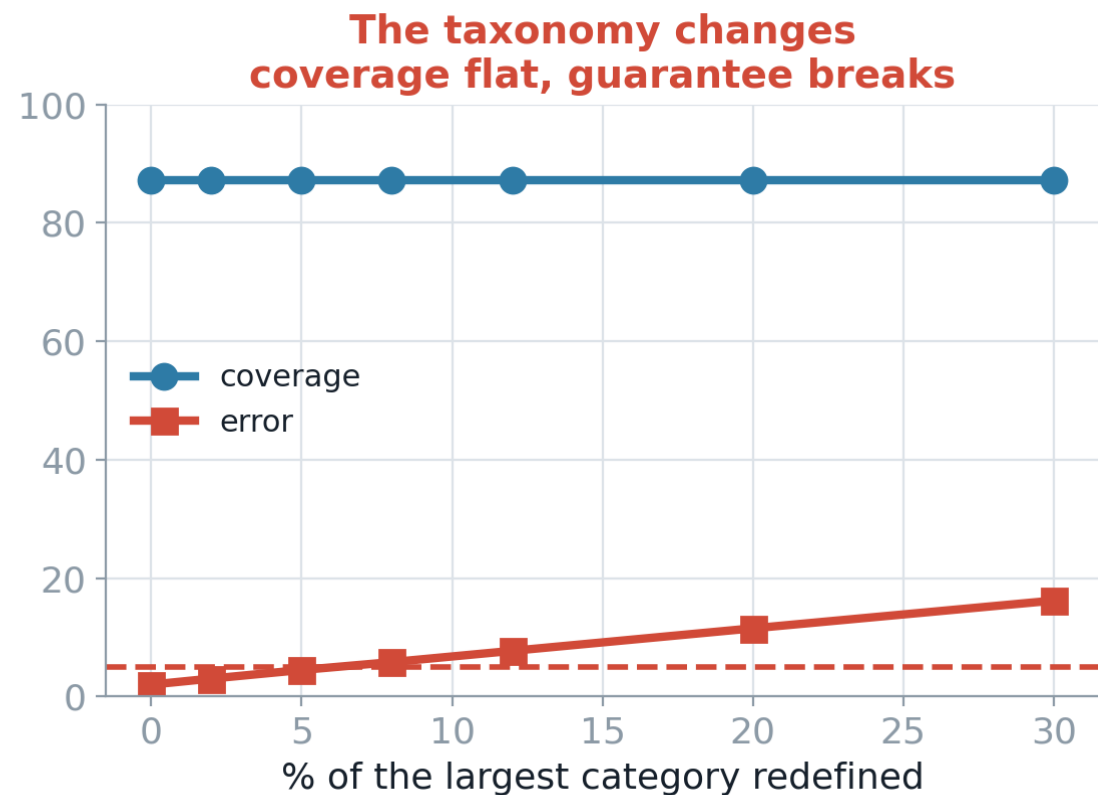
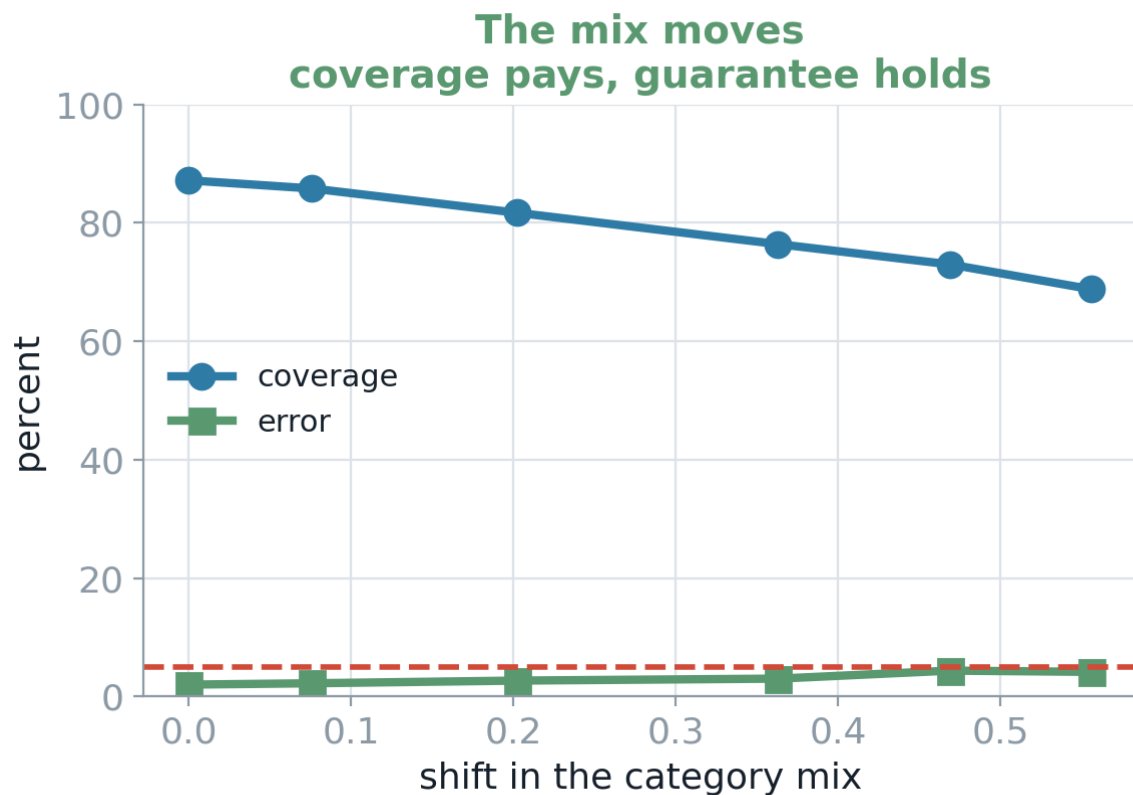
317

labelled items a category needs before any guarantee can be certified for it

At the 97% target, none of the 43 certify at all.

This is the number the proposal needed: Phase 0 should size the labelling exercise at roughly 300 examples per category to be automated, not by a global sample target.

Two kinds of drift. Only one is visible.



Left: the guarantee survives an extreme shift because unfamiliar tickets score low and get held back. Right: coverage does not move by a tenth of a point while the error rate triples.

Coverage is not a health signal

Nothing inside the service detects a taxonomy change. It breaks the guarantee once 8% of one category is redefined, with every internal metric flat.

The only thing that catches it is ground truth coming back from production: agents re-routing tickets the system routed automatically.

In the proposal, feedback capture moved from a refinement to a requirement.

Re-route rate per category becomes the primary production health metric, and the trigger for recalibrating ahead of schedule.

8%

of one high-volume category redefined is enough to break a certified guarantee

87.2%

coverage before, during and after. It never moves.

Three decisions made by measurement

Hoeffding → empirical Bernstein

The first bound could not certify anything below a 10% error target, refusing the exact operating point the exercise was about. It charges for the full range regardless of observed spread; this loss is almost always 0 or alpha.

Recovered the 95%, 97% and 98% targets at the same rigour.

Bonferroni → fixed sequence testing

Testing 200 grid points at $\delta/200$ pays a multiplicity price for grid resolution, which is an implementation detail the data has no stake in.

Bought 4 to 9 points of coverage. But it is not universally better, and the harness runs both.

Multi-start: tried and rejected

The variant splits the budget across several entry points to survive non-monotone bumps in the risk.

Splitting the budget cost more than the bumps did. The code is left in place, unused, with the result recorded.

Each was a measurement, not a preference. The one that did not work is on the slide too.

Three things I got wrong

All three were caught by the test suite, not by looking at the output. The numbers looked perfectly reasonable while they were wrong.

The finding I retracted

I reported that the guarantee broke at a specific amount of mix shift, and recommended monitoring that distance in production. With the bugs fixed it survives twice as far. The recommendation was wrong, and the replacement finding is the opposite in character.

Two bugs behind it

Categories with few documents were getting no exemplar in the retrieval index, and the corpus carries 85 exact duplicate documents, one appearing seven times. A duplicate straddling index and test lets the classifier retrieve a copy of the item it is classifying.

A third, worse one

Two definitions of the per-category function existed; the later one, mine, failed to split the confidence budget across classes. Python keeps the last definition, so the statistically incorrect one was the one running.

A measurement you cannot check is not a measurement. This is why the test suite exists, and why it is in the report.

What this is not



The encoder is TF-IDF, not a sentence model

The build environment had no access to pretrained transformer weights. It has no semantic knowledge beyond term co-occurrence in this corpus and cannot handle a multilingual flow at all. Swapping it moves every accuracy number here. It does not change whether the threshold selection is valid, which is what was being tested.



Reuters is easier than support tickets

Newswire is well written, consistently labelled and monolingual. A published project on Russian-language IT support with roughly a million training tickets reached 84% precision at 61% coverage, against human operators at 75%.



The conformal work is application, not contribution

The methods are from the literature and cited. What is mine is the implementation, the measurement, and the decisions between variants.



Label noise is not modelled

Every corpus label is treated as correct. In a real archive that assumption is usually false, which is precisely why measuring it is Phase 0 of the proposal rather than an assumption inside it.

What the client would actually get

1

A target that means something

Precision chosen by the client, coverage reported as the measured consequence, per category rather than as one aggregate.

2

A number proven before go-live

Four weeks of shadow mode on live traffic, writing nothing, before automated routing touches a ticket.

3

A system that says no

It refuses to certify what the data cannot support, and every decision carries the archive documents behind it.

Report, source, tests and the interactive version: github.com/thomas2143